

programación orientada a objetos

en Scala

principios de la programación orientada a objetos

- la programación orientada a objetos organiza el código en objetos que combinan datos y comportamiento
- un objeto encapsula su estado interno y expone operaciones bien definidas
- los objetos colaboran entre sí enviándose mensajes (llamando métodos)
- los cuatro pilares de la POO:
 - → encapsulación: ocultar el estado interno
 - → herencia: reutilizar y extender comportamiento
 - → polimorfismo: tratar distintos tipos de forma uniforme
 - → abstracción: modelar con clases e interfaces

POO en Scala

- Scala es un lenguaje orientado a objetos puro: todo valor es un objeto
- incluso los tipos primitivos (Int, Boolean) son objetos
- incluso las funciones son objetos (Function1, Function2, ...)
- provee clases, clases abstractas, traits, herencia y polimorfismo
- el compilador verifica la jerarquía de tipos en tiempo de compilación
- combina POO y programación funcional de forma natural

clases

- una clase define la estructura y el comportamiento de sus instancias
- los parámetros del constructor se declaran directamente en la firma de la clase
- los métodos son funciones definidas dentro de la clase
- los atributos pueden ser val (inmutables) o var (mutables)

classes — Java vs. Scala

Java

```
public class Person {  
    private String name;  
    private int age;  
  
    public Person(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
    public String getName() { return name; }  
    public int getAge()    { return age; }  
}
```

Scala

```
class Person(val name: String, val age: Int)
```

```
// uso:
```

```
val p = new Person("Alan Turing", 41)  
println(p.name) // Alan Turing  
println(p.age)  // 41
```

clases abstractas

- una clase abstracta no puede instanciarse directamente
- define estructura y comportamiento común para sus subclasses
- puede tener métodos abstractos (sin implementación) y concretos (con implementación)
- las subclasses deben implementar todos los métodos abstractos
- permite usar el patrón template method: métodos concretos que invocan abstractos

clases abstractas

```
abstract class NamedEntity(val text: String) {
```

```
    // método abstracto: cada subclase lo implementa  
    def entityType: String
```

```
    // método concreto: usa entityType → funciona para cualquier subclase  
    def describe: String = s"[$entityType] $text"  
}
```

```
class Person(text: String) extends NamedEntity(text) {  
    def entityType: String = "Person"  
}
```

```
val p = new Person("Alan Turing")  
println(p.describe) // [Person] Alan Turing  
println(p.entityType) // Person
```

herencia

- una subclase hereda todos los atributos y métodos de su clase padre
- se declara con la palabra clave `extends`
- la subclase puede redefinir métodos con `override`
- permite reutilizar código y expresar relaciones conceptuales entre tipos
- Scala solo admite herencia simple de clases (una sola clase padre)
- pero permite múltiples traits (ver siguiente sección)

herencia

```
abstract class NamedEntity(val text: String) {  
    def entityType: String  
    def describe: String = s"[$entityType] $text"  
}
```

```
class Organization(text: String) extends NamedEntity(text) {  
    def entityType: String = "Organization"  
}
```

```
// University hereda de Organization y redefine entityType  
class University(text: String) extends Organization(text) {  
    override def entityType: String = "University"  
}
```

```
val u = new University("MIT")  
println(u.describe) // [University] MIT  
println(u.entityType) // University
```

traits

- un trait define un contrato (interfaz) que las clases pueden implementar
- a diferencia de las clases abstractas, un trait no tiene constructor propio
- una clase puede mezclar múltiples traits (with)
- los traits pueden tener métodos abstractos y concretos
- son la herramienta principal para compartir comportamiento entre clases no relacionadas
- similares a las interfaces de Java 8+ (con default methods)

traits

```
trait Describable {  
  def describe: String      // abstracto  
  def print(): Unit = println(describe) // concreto  
}
```

```
trait Countable {  
  def count: Int  
}
```

```
class Organization(val name: String)  
  extends NamedEntity(name)  
  with Describable  
  with Countable {  
  
  def entityType = "Organization"  
  def describe   = s"[Organization] $name"  
  def count      = 1  
}
```

polimorfismo

- polimorfismo: un mismo método se comporta de forma distinta según el tipo del objeto
- permite operar sobre una lista heterogénea de objetos de forma uniforme
- no es necesario hacer if o match sobre el tipo concreto
- el método correcto se elige en tiempo de ejecución (dispatch dinámico)
- es la consecuencia natural de combinar herencia con redefinición de métodos

polimorfismo

sin polimorfismo

```
def printEntity(e: NamedEntity): Unit = {  
  e match {  
    case _: Person =>  
      println(s"Persona: ${e.text}")  
    case _: University =>  
      println(s"Universidad: ${e.text}")  
    case _: ProgrammingLanguage =>  
      println(s"Lenguaje: ${e.text}")  
    // hay que actualizar esto  
    // cada vez que se agrega un tipo...  
  }  
}
```

con polimorfismo

```
def printEntity(e: NamedEntity): Unit =  
  println(e.describe)  
  // describe está definido en NamedEntity  
  // cada subclase provee su entityType  
  // no hay que modificar este método  
  // al agregar nuevos tipos  
  
val entities: List[NamedEntity] = List(  
  new Person("Alan Turing"),  
  new University("MIT"),  
  new ProgrammingLanguage("Scala")  
)  
entities.foreach(e => println(e.describe))
```

objetos singleton

```
// object reemplaza los métodos estáticos de Java
// es una instancia única (singleton) de su propio tipo
object Dictionary {
```

```
  def loadAll(): List[NamedEntity] = {
    loadFromFile("data/people.txt", "Person") :::
    loadFromFile("data/languages.txt", "ProgrammingLanguage")
  }
```

```
  def loadFromFile(path: String, entityType: String): List[NamedEntity] = {
    // ...
  }
}
```

```
// uso: sin new, se accede directamente
val dict = Dictionary.loadAll()
```

equivale a una clase con todos los miembros estáticos en Java

POO en Scala — ejemplos

jerarquía de entidades — Lab 2

```
abstract class NamedEntity(val text: String) {  
  def entityType: String  
  def describe: String = s"[$entityType] $text"  
}
```

```
class Person(text: String) extends NamedEntity(text) { def entityType = "Person" }  
class Organization(text: String) extends NamedEntity(text) { def entityType = "Organization" }  
class University(text: String) extends Organization(text){ override def entityType = "University" }  
class Place(text: String) extends NamedEntity(text) { def entityType = "Place" }  
class Technology(text: String) extends NamedEntity(text) { def entityType = "Technology" }  
class ProgrammingLanguage(text: String) extends Technology(text) {  
  override def entityType = "ProgrammingLanguage"  
}
```


polimorfismo en el lab

// una lista heterogénea de entidades

```
val entities: List[NamedEntity] = List(  
  new Person("Martin Odersky"),  
  new University("EPFL"),  
  new ProgrammingLanguage("Scala")  
)
```

// describe funciona correctamente para todos los tipos

```
entities.foreach(e => println(e.describe))
```

```
// [Person] Martin Odersky
```

```
// [University] EPFL
```

```
// [ProgrammingLanguage] Scala
```

// contar por tipo — sin ningún match

```
val counts = entities.groupBy(_.entityType).view.mapValues(_.size).toMap
```

```
// Map("Person" -> 1, "University" -> 1, "ProgrammingLanguage" -> 1)
```

imperativo vs. orientado a objetos

sin clases

```
// datos como tuplas o mapas
type Entity = (String, String)
//      text, type

def describe(e: Entity): String =
  s"${e._2}] ${e._1}"

def countByType(
  es: List[Entity]
): Map[String, Int] =
  es.groupBy(_._2)
    .view.mapValues(_.size).toMap

// el tipo y el comportamiento
// están separados
```

con clases

```
// datos + comportamiento juntos
abstract class NamedEntity(val text: String) {
  def entityType: String
  def describe: String =
    s"[$entityType] $text"
}

class Person(text: String)
  extends NamedEntity(text) {
  def entityType = "Person"
}

// agregar un tipo nuevo =
// agregar una clase,
// sin tocar el resto del código
```

funcional vs. orientado a objetos

- paradigma funcional (Lab 1):
 - → funciones que transforman datos
 - → composición de funciones
 - → datos separados del comportamiento
- paradigma orientado a objetos (Lab 2):
 - → objetos que encapsulan datos y comportamiento
 - → jerarquías de clases
 - → colaboración entre objetos
- Scala permite combinar ambos enfoques según el problema

materiales de referencia

- <https://docs.scala-lang.org/tour/classes.html>
- <https://docs.scala-lang.org/tour/traits.html>
- <https://docs.scala-lang.org/tour/polymorphism.html>
- <https://docs.scala-lang.org/tour/abstract-type-members.html>